

类与对象

复习

- 1.生成54张牌
- 2.洗牌
- 3.发牌

目录

- | | |
|---------------------|----|
| 1. 面向对象和面向过程的区别 | 了解 |
| 2. 类与对象的基本概念 | 了解 |
| 3. 类class与类对象 | 熟悉 |
| 4. 构造函数与析构函数 | 重点 |
| 5. 动态创建对象new/delete | 重点 |
| 6. 动态创建对象数组 | 重点 |
| 7. 成员变量与成员函数 | 重点 |
| 8. 静态成员变量与静态成员函数 | 重点 |
| 9. const修饰类的成员函数 | 熟悉 |
| 10. this指针 | 重点 |

面向对象和面向过程的区别

面向过程就是分析出解决问题所需要的步骤，然后把这些步骤一步一步实现的过程。

面向对象是把构成问题事务分解成各个对象，建立对象的目的不是为了完成一个步骤，而是为了描述某个事物在整个解决问题的步骤中的行为。

面向对象编程就是高度实物抽象化编程！

面向过程编程就是自顶向下按步骤编程！

面向对象和面向过程的区别

例如实现一个番茄炒鸡蛋！

面向过程：

先买番茄和鸡蛋->洗番茄->搅拌鸡蛋->生火->炒菜->番茄炒鸡蛋

面向对象：

告诉厨师想吃番茄炒鸡蛋->厨师->番茄炒鸡蛋

面向对象和面向过程的区别

面向过程：

优点:性能比面向对象高，因为类调用时需要实例化，开销比较大，比较消耗资源。而Linux\Unix等一般采用面向过程开发，性能是最重要的因素。

缺点:没有面向对象易维护，易复用，易扩展。可维护性差，不易修改。

面向对象：

优点:易维护，易复用，易扩展。由于面向对象由封装，继承，多态性的特性，可以设计出耦合度低的系统，使系统更加灵活，更加易于维护。

缺点:性能比面向过程低。

类与对象的基本概念

什么是类

具有相同特性（数据元素）和行为（功能）的对象的抽象就是类。因此，对象的抽象是类，类的具体化就是对象，也可以说类的实例是对象，类实际上就是一种数据类型。

类具有属性，它是对象的状态的抽象，用数据结构来描述类的属性。类具有操作，它是对象的行为的抽象，用操作名和实现该操作的方法来描述。

什么是对象

对象是人們要進行研究的任何事物，它不僅能表示具體的事物，還能表示抽象的規則、計劃或事件。對象具有狀態，一個對象用數據值來描述它的狀態。對象還有操作，用於改變對象的狀態，對象及其操作就是對象的行為。

對象實現了數據和操作的結合，使數據和操作封裝於對象的統一體中。

类与对象的基本概念

类



对象



类class

定义

在 C++ 中，定义类，使用 **class** 关键字。类名的首字母一般都大写。

语法

class ClassName

{

PropertyModifier:

 propertoys

 functions

};

class

定义类使用的关键字。

ClassName

类名。

PropertyModifier 属性修饰符，有 public、protected、private。

propertoys

属性列表。

functions

函数列表。

类里面的属性列表和函数列表需要使用 {} 包围，同时，类的最后一定不能忘记**分号**。

类class

定义一个学生类，拥有打印自己姓名的方法。

```
class Student
{
public:
    void printName(string name)
    {
        cout << name << endl;
    }
};
```

类对象

使用类创建对象的过程，又称为类的实例化。

语法

Classname classVar(param);

类名 类对象 (参数);

类对象

```
class Student
{
public:
    void printName(string name)
    {
        cout << name << endl;
    }
};

int main()
{
    Student stu; //实例化Student类的对象stu
    return 0;
}
```

构造函数与析构函数

构造函数

创建一个类时，会自动添加一个跟类名一样的函数，该函数没有任何的返回值，且该函数不做任何操作，该函数被称为构造方法（或构造函数）。

构造方法用于创建对象时使用，每当创建一个类的实例对象时，C++ 解释器都会自动调用它。

语法

```
class Classname
{
public:
    Classname(){}
};
```

构造函数与析构函数

构造函数

创建一个类时，会自动添加一个跟类名一样的函数，该函数没有任何的返回值，且该函数不做任何操作，该函数被称为构造方法（或构造函数）。

构造方法用于创建对象时使用，每当创建一个类的实例对象时，C++ 解释器都会自动调用它。

语法

```
class Classname
{
public:
    Classname(){}
};
```

构造函数与析构函数

```
class Student
{
public:
    Student(){
        cout << "Student()" << endl;
    }
    void printName(string name)
    {
        cout << name << endl;
    }
};
```

构造函数参数

C++ 中的构造函数，默认是没有任何函数参数的，但我们也可以重写该函数，为其设置任意多个和任意类型的参数，如果，我们在构造函数方法里设置了参数，那么我们在实例化类对象的时候，必须要传入相对应的参数。

C++ 的构造函数，跟普通的函数一样，也可以设置默认参数，如果设置了默认参数，那么在实例化实例时，可以不传该参数，而使用默认值。

构造函数参数

```
class Student
```

```
{
```

```
public:
```

```
    Student(string name,int age){
```

```
        cout << "Student()  " << name << " " << age << endl;
```

```
    }
```

```
    void printName(string name)
```

```
    {
```

```
        cout << name << endl;
```

```
    }
```

```
};
```

拷贝构造函数

拷贝构造函数也叫复制构造函数，它只有一个参数，参数类型是本类的引用。参数可以是 `const` 引用，也可以是非 `const` 引用。一般使用前者，这样既能以常量对象（初始化后值不能改变的对象）作为参数，也能以非常量对象作为参数去初始化其他对象。一个类中写两个复制构造函数，一个的参数是 `const` 引用，另一个的参数是非 `const` 引用，也是可以的。

如果类的设计者不写复制构造函数，编译器就会自动生成复制构造函数。大多数情况下，其作用是实现从源对象到目标对象逐个字节的复制，即使得目标对象的每个成员变量都变得和源对象相等。编译器自动生成的复制构造函数称为“默认复制构造函数”。

注意，默认构造函数（即无参构造函数）不一定存在，但是复制构造函数总是会存在。

拷贝构造函数

拷贝构造函数通常用于：

1. 通过使用另一个同类型的对象来初始化新创建的对象。
2. 复制对象把它作为参数传递给函数。
3. 复制对象，并从函数返回这个对象。

拷贝构造函数

```
class Complex
{
public:
    double real, imag;
    Complex(double r, double i) {
        real= r; imag = i;
    }
};

int main(){
    Complex c1(1, 2);
    Complex c2 (c1); //用复制构造函数初始化c2
    cout<<c2.real<<" "<<c2.imag; //输出 1,2
    return 0;
}
```



默认拷贝构造函数

拷贝构造函数

```
class Complex{
public:
    double real, imag;
    Complex(double r,double i){
        real = r;
        imag = i;
    }
    Complex(const Complex & c){
        real = c.real;
        imag = c.imag;
        cout<<"我被调用了"<<endl;
    }//编写了复制构造函数，则默认复制构造函数就不存在了
};

int main(){
    Complex c1(1, 2);
    Complex c2 (c1); //调用复制构造函数
    cout<<c2.real<<"    "<<c2.imag;
    return 0;
}
```



拷贝构造函数

构造函数与析构函数

析构函数

在类被销毁时，系统还提供了一个析构函数，用来做一些清理工作，比如释放分配的内存、关闭打开的文件等，同样，析构函数，也是系统自动调用。

析构函数（Destructor）也是一种特殊的成员函数，没有返回值，不需要程序员显式调用（程序员也没法显式调用），而是在销毁对象时自动执行。构造函数的名字和类名相同，而析构函数的名字是在类名前面加一个~符号。

析构函数**没有参数，不能被重载**，因此一个类只能有一个析构函数。如果用户没有定义，编译器会自动生成一个默认的析构函数。

析构函数

语法

```
~Destructor()  
{  
}
```

析构函数的前面一定要加 ~，并且是系统自动调用的。

析构函数

```
class Student
{
public:
    Student(string name, int age)
    {
    }

    ~Student()
    {
        cout << "Call ~Student" << endl;
    }
};
```

动态创建对象new/delete

当创建一个C++对象时，会发生两件事：

- 1 为对象分配内存
- 2 调用构造函数来初始化那块内存

在步骤1中，可以用几种方式或在可选择的时间发生：

- ① 在静态存储区域，存储空间在程序开始之前就可以分配。
- ② 无论何时到达一个特殊的执行点（左大括号）时，存储单元都可以在栈上被创建。出了执行点（右大括号），这个存储单元自动被释放。
- ③ 存储单元也可以从一个称为堆（自由存储单元）的地方分配。这被称为动态内存分配。

第1步能保证实现，需要确保第2步一定能发生。c++强迫这么做是因为使用未初始化的对象是程序出错的一个重要原因。

new操作符

C++中解决动态内存分配的方案是把创建一个对象所需要的操作都结合在一个称为new的运算符里。当用new创建一个对象时，它就在堆里为对象分配内存并调用构造函数完成初始化。

它有点类似于C里面的malloc函数；C的标准库中提供了一些函数,malloc以及它的变种calloc和realloc,释放内存的free,这些函数是有效的、但是原始的，需要程序员理解和小心使用。

C的动态内存分配

```
class Person{  
public:
```

使用C的动态内存分配函数在堆上创建
一个类的实例，我们必须这样做

```
    Person(){  
        age = 20;  
        pName = (char*)malloc(strlen("jack")+1);  
        strcpy(pName, "jack");  
    }  
    void Init(){  
        age = 20;  
        pName = (char*)malloc(strlen("jack")+1);  
        strcpy(pName, "jack");  
    }  
    void Clean(){  
        if (pName != NULL){  
            free(pName);  
        }  
    }  
public:  
    int age;  
    char* pName;  
};
```

```
int main(){  
    //分配内存  
    Person* person = (Person*)malloc(sizeof(Person));  
    if(person == NULL){  
        return 0;  
    }  
    //调用初始化函数  
    person->Init();  
    //清理对象  
    person->Clean();  
    //释放person对象  
    free(person);  
  
    return 0;  
}
```

C的动态内存分配

- 程序员必须确定对象的长度。
- malloc返回一个void*指针，c++不允许将void*赋值给其他任何指针，必须强转。
- malloc可能申请内存失败，所以必须判断返回值来确保内存分配成功。
- 用户在使用对象之前必须记住对他初始化，构造函数不能显示调用初始化(构造函数是由编译器调用)，用户有可能忘记调用初始化函数。

C的动态内存分配函数太复杂，容易令人混淆，是不可接受的，c++中我们推荐使用运算符new 和 delete.

new 操作符

```
Person* person = new Person();
```

等价于:

```
Person* person = (Person*)malloc(sizeof(Person));
```

```
if(person == NULL){
```

```
    return 0;
```

```
}
```

```
person->Init();
```

New操作符能确定在调用构造函数初始化之前内存分配是成功的，所有不用显式确定调用是否成功。

delete操作符

delete表达式先调用析构函数，然后释放内存。

正如new表达式返回一个指向对象的指针一样，delete需要一个对象的地址。

new表达式的反面是delete表达式。

delete只适用于由new创建的对象。

如果使用一个由malloc或者calloc或者realloc创建的对象使用delete,这个行为是未定义的。因为大多数new和delete的实现机制都使用了malloc和free,所以很可能没有调用析构函数就释放了内存。

如果正在删除的对象的指针是NULL,将不发生任何事，因此建议在删除指针后，立即把指针赋值为NULL，以免对它删除两次，对一些对象删除两次可能会产生某些问题。

delete操作符

```
Person* person1 = new Person;
```

```
delete person1;
```

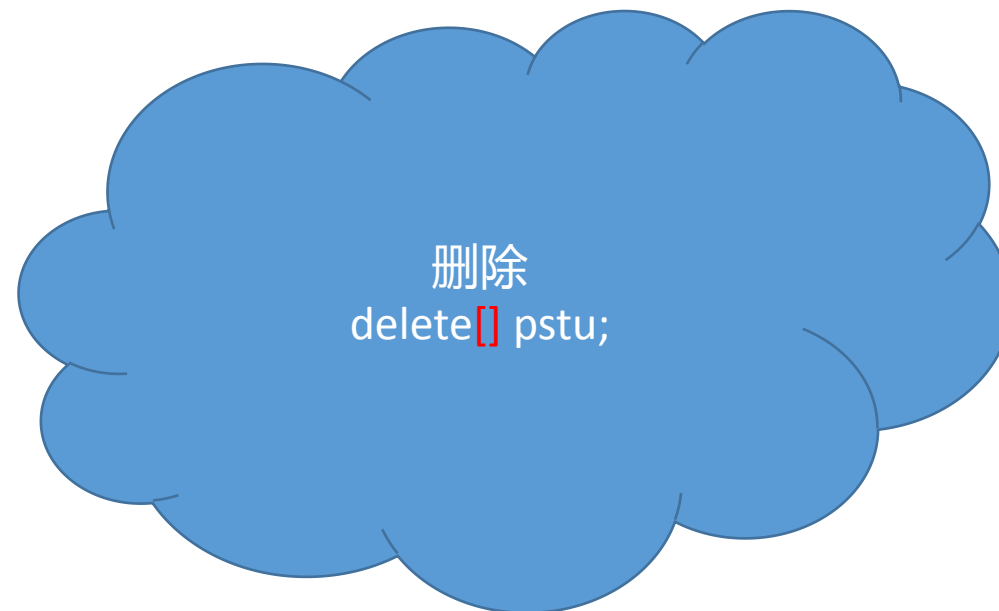
动态创建对象数组

当创建一个对象数组的时候，必须对数组中的每一个对象调用构造函数，除了在栈上可以聚合初始化，必须提供一个默认的构造函数。

动态创建对象数组

```
class Student{
public:
    Student(){//栈上可以聚合初始化，堆上创建必须提供一个默认的构造函数
        cout << 111<< endl;
    }
    Student(string name,int age){
        cout << name << endl;
        cout << age<< endl;
    }
    void func1(){
        cout << "func1" << endl;
    }
};

int main(){
    //栈聚合初始化
    Student stu[] = {"zs",20},{"ww",21};
    //创建堆上对象数组必须提供构造函数
    Student *pstu = new Student[20];
    return 0;
}
```



new/delete用于普通数组

//创建字符数组

```
char* p1 = new char[50];
```

//创建整型数组

```
int* p2 = new int[100];
```

//创建整型数组并初始化

```
int* p3 = new int[10]{ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
```

//释放数组内存

```
delete[] p1;
```

```
delete[] p2;
```

```
delete[] p3;
```

成员变量与成员函数

C++ 类的内部定义的 变量 被称为成员变量。类的成员变量可以定义 0 到多个。

语法

```
class ClassName
{
public:
    Type paramter;
};
```

成员变量的访问只能使用对象访问。

instance.paramter;

成员变量与成员函数

```
class Student
{
public:
    Student()
    {
    }
    string name;//成员变量
};
```

成员变量与成员函数

C++ 类的普通成员函数是定义在类内部的方法，C++ 的成员方法与普通函数一样，可以有任意的参数和返回值，同时，也支持默认参数和函数重载。

语法

```
class ClassName
{
public:
    Type funcName(paramterType paramter)
    {
    }
};
```

通过“实例.方法名”的方法，调用成员方法。

```
instance.funcName(params)
```

成员变量与成员函数

```
class Student
{
public:
    void printName(string name)//成员函数
    {
        cout << name << endl;
    }
};
```

成员变量与成员函数 案例

```
#include <iostream>
using namespace std;
class Student{
public:
    void printName(){
        cout << name << endl;
    }
public:
    string name;
    int age;
};
int main(){
    Student s1;
    s1.name = "张三";
    s1.printName();
    return 0;
}
```

静态成员变量与静态成员函数

在 C++ 中，**类的静态成员**可以实现多个对象之间的数据共享，并且使用静态数据成员还不会破坏隐藏的原则，即保证了安全性。因此，静态成员是类的所有对象中共享的成员，而不是某个对象的成员。

使用静态数据成员可以节省内存，因为它是所有对象所公有的，因此，对多个对象来说，静态数据成员只存储一处，供所有对象共用。

静态数据成员的值对每个对象都是一样，但它的值是可以更新的。只要对静态数据成员的值更新一次，保证所有对象存取更新后的相同的值，这样可以提高时间效率。

在访问类的静态成员时，**既可以使用类名来访问也可以使用对象名来访问。**

静态成员变量

定义

静态数据成员在定义或说明时前面加 **关键字 static**。

语法

```
class ClassName
{
    public:
        static Type paramter;
};
```

静态成员初始化

定义

静态成员在定义后，必须要进行初始化。

语法

```
Type ClassName::paramter=value;
```

静态成员引用

定义

静态成员的引用可以使用类名或者对象名引用。

语法

`ClassName::paramter`

静态成员变量案例

```
class Student
{
public:
    static string course;
};

string Student::course = "C++";

int main()
{
    cout << "Student::course = " << Student::course << endl;
    return 0;
}
```

静态成员函数

C++ 类的静态成员方法的需要使用 **static 来修饰**，且 C++ 类的静态成员方法与 静态成员变量 类似，静态成员方法不属于任何对象，只属于类。同时，**C++ 的静态成员函数只能访问静态成员。**

静态成员函数可以通过类来直接调用，编译器不会为它增加形参 `this`，它不需要当前对象的地址，所以不管有没有创建对象，都可以调用静态成员函数。

静态成员函数

定义

```
class ClassName
```

```
{
```

```
public:
```

```
    static Type funcName(paramterType paramter)
```

```
    {
```

```
    }
```

```
};
```

静态成员方法调用

定义

C++ 的静态方法调用方法有两种，一种是通过 “**类名::方法名**” 的方法来调用，另一种是通过 “**对象.方法名**” 的方法来调用。

语法

ClassName::funcName(params)

instance.funcName(params)

静态成员方法调用 案例

```
class Student
{
public:
    Student()
    {
    }
    static void sayHello()
    {
        cout << "Hello C++!" << endl;
    }
};

int main()
{
    Student::sayHello();
    Student stu;
    stu.sayHello();
    return 0;
}
```

const修饰类的成员函数

在 C++ 中，const 关键字除了可以修饰 变量 和 成员变量 之外，还可以修饰 成员函数，使用 const 修饰的成员函数，叫做**常成员函数**。

常成员函数需要在声明和定义的时候在函数头部的结尾加上 const 关键字，常成员函数，**可以访问类中的所有的成员变量，但不可以修改他们的值**，常成员函数主要是为了**保护数据而设置**的。

const修饰类的成员函数

定义

常成员函数需要在声明和定义的时候在函数头部的结尾加上 const 关键字。

语法

```
class ClassName
{
public:
    type func() const
    {
    }
};
```

const修饰类的成员函数

```
class Student
{
public:
    Student(string name, int age)
    {
        this->m_name = name;
        this->m_age = age;
    }
    void sayHello() const
    {
        cout << "I am " << this->m_name << " and i am " << this->m_age << " years old!" << endl;
    }
private:
    string m_name;
    int m_age;
};

int main()
{
    Student stu("张三丰", 110);
    stu.sayHello();
    return 0;
}
```

this指针

在 C++ 中，**有一个 this 关键字**，其是一个指针，同时也是一个 const 指针，它指向当前对象（也就是当前正在使用的对象），通过它可以访问当前对象的所有成员。

C++ this指针的本质

this 实际上是 成员函数 的一个形参，在调用成员函数时将对象的地址作为实参传递给 this。不过 this 这个形参是隐式的，它并不出现在代码中，而是在编译阶段由编译器默默地将它添加到参数列表中。

this 作为隐式形参，本质上是成员函数的局部变量，所以只能用在成员函数的内部，并且只有在通过对象调用成员函数时才给 this 赋值。

this 是 const 指针，它的值是不能被修改的，一切企图修改该指针的操作，如赋值、递增、递减等都是不允许的。

this 只能在成员函数内部使用，用在其他地方没有意义，也是非法的。只有当对象被创建后 this 才有意义，因此不能在 static 成员函数中使用。

this指针 案例

```
class Student
{
public:
    Student(string name, int age)
    {
        this->m_name = name;
        this->m_age = age;
    }
    void show()
    {
        cout << "this = " << this << endl;
    }
private:
    string m_name;
    int m_age;
};

int main()
{
    Student stu("zsf", 120);
    stu.show();
    cout << "stu = " << &stu << endl;
    Student stu1("zsf2", 200);
    stu1.show();
    cout << "stu1 = " << &stu1 << endl;
    return 0;
}
```

练习

创建一个矩形类，成员变量有长和宽，成员函数有判断是否是正方形和输出矩形的面积；创建实例对象，调用函数判断是否是正方形和打印面积。

练习 代码

```
#include <iostream>
using namespace std;
class Rect{
private:
    double width;
    double height;
public:
    Rect(double width,double height){
        this->width = width;
        this->height = height;
    }
    bool is_square(){
        return this->width == this->height;
    }
    void area(){
        cout << this->width * this->height << endl;
    }
};

int main(){
    Rect r1 = {10,20};
    bool b = r1.is_square();
    if(b){
        cout << "是正方形" << endl;
    }
    else{
        cout << "不是正方形" << endl;
    }
    r1.area();
    return 0;
}
```

知识点总结

- | | |
|---------------------|----|
| 1. 面向对象和面向过程的区别 | 了解 |
| 2. 类与对象的基本概念 | 了解 |
| 3. 类class与类对象 | 熟悉 |
| 4. 构造函数与析构函数 | 重点 |
| 5. 动态创建对象new/delete | 重点 |
| 6. 动态创建对象数组 | 重点 |
| 7. 成员变量与成员函数 | 重点 |
| 8. 静态成员变量与静态成员函数 | 重点 |
| 9. const修饰类的成员函数 | 熟悉 |
| 10. this指针 | 重点 |

作业

1. 定义一个动物类，实现一个静态变量count记录类被实例化的次数，创建动物的吃、跑、睡等动作，在创建两个实例，来分别调用。

下节课预告

访问限定符、运算符重载等